PungentFunk Utilities - Architecture and Design Principles Bible

Verifiable UI, Source-Truth, and Resizable Panel Doctrine Edition

PungentFunk Utilities

2026-05-08

Table of Contents

PungentFunk Utilities - Architecture and Design Principles Bible

Edition: Verifiable UI, Source-Truth, and Resizable Panel Doctrine
Purpose: Stable architecture, product design, Unity Foundations alignment, editor implementation doctrine, AI-agent interpretation rules, verifiable UI preservation standards, and package-wide resizable panel/layout implementation rules.
Update rule: Update this bible when product philosophy, package boundaries, UI doctrine, interoperability rules, source-truth rules, AI-agent interpretation rules, or documentation governance changes. Update snapshot audits separately whenever scripts change.

What changed in this edition

This edition keeps the manual doctrine spine and the AI-agent interpretation layer, then strengthens the rules that should have prevented recent cleanup regressions. It adds a verifiability layer for implementation claims, UI relocation, visual state, feature reachability, split-panel behavior, adaptive row wrapping, contextual help, generated documentation, and persistent editor view state.

The core correction is this:

- A debrief is not implementation evidence.
- A summary that says a section was moved or restored is not proof that the active UI exposes it correctly.
- A source file is not enough if the feature is unreachable, hidden behind the wrong condition, duplicated, cramped, or visually unusable.
- A UI cleanup is a regression if it hides functionality users still need.
- A panel implementation is incomplete if resize handles, clamping, stretching, wrapping, help context, or persisted state only work after tool-specific cleanup passes.

From this edition forward, an update pass is only successful when the current source structure and the active UI surface both verify the claim.

0. How to use this bible

What this document is

This is the doctrine source for PungentFunk Utilities. It records:

- architecture and package boundaries;
- Unity Editor implementation doctrine;
- UI, accessibility, layout, and messaging principles;
- dependency, optional integration, and project adapter rules;
- release-hardening expectations;
- AI-agent interpretation rules for audits, briefs, and implementation passes;
- verifiable UI preservation rules for source-backed, visible, reachable, non-regressive updates.

It does **not** record:

- volatile script counts;
- current feature inventory status;
- sprint tasks or one-off bug lists;
- per-PR implementation details;
- tutorial-level API instructions;
- exhaustive code examples.

Reading paths

- **AI coding agent preparing an audit:** Read sections 0, 2, 3, 4, 8, 9, 12, 17, 18, and the current snapshot audit.
- **AI coding agent preparing an implementation brief:** Read sections 0, 2, 3, 4, 6, 7, 8, 9, 10, 17, and 18, especially the shared resizable panel contract for UI work.
- **AI coding agent editing code:** Read this bible, the project skill, path-specific instructions, the current source files, and the active UI structure before proposing changes.
- **Human implementer:** Read sections 1, 2, 4, 7, 8, 9, 13, and the relevant lab spec or pattern page.
- **Reviewer:** Read sections 3, 4, 8, 12, 17, 18, 19, and the changed files.
- **Designer or UX contributor:** Read sections 4, 5, 6, 8, 9, and the visual examples or pattern pages, especially panel sizing, wrapping, and help-affordance rules.
- **Producer or planner:** Read sections 0, 1, 3, 16, 18, 19, and the current roadmap.

Source-of-truth hierarchy

When documents conflict, apply this priority order:

1. The user's current explicit instruction.
2. Current uploaded source files or the current repository source tree.

3. The active UI as rendered in Unity, including visibility conditions, layout, scroll behavior, resizing behavior, and interactive reachability.
4. This design bible for durable doctrine.
5. The current snapshot audit for implementation state.
6. The current feature inventory for backlog/status.
7. Lab specs and pattern pages for detailed implementation guidance.
8. Historical briefs, older audits, old debriefs, and conversation notes.

AI agents must not treat old snapshot counts, old feature inventories, old implementation briefs, or debrief summaries as current implementation facts when a newer archive or current source tree is available.

Implementation-state vocabulary

Use these terms precisely:

- **Implemented in source:** Relevant classes, fields, methods, registrations, and call paths exist in current files.
- **Reachable in UI:** A user can access the feature through the intended menu, window, panel, inspector, context menu, overlay, or shortcut without hidden prerequisites.
- **Visible in UI:** The feature is displayed in the intended surface at usable sizes and is not obscured, clipped, buried under unrelated sections, or only visible after accidental layout expansion.
- **Usable in UI:** The controls are legible, interactable, not cramped beyond reasonable use, and provide expected feedback.
- **Verified:** Source evidence, active UI evidence, and validation steps all support the claim.

Do not mark a feature as fully complete unless it is implemented in source, reachable, visible, usable, and verified.

1. Scope and North Star

PungentFunk Utilities is a family of modular Unity editor/runtime utility assets, not a single monolithic toolbox. Each lab or suite should be independently releasable, while the larger bundle should expose a cohesive control panel, shared visual language, consistent interaction vocabulary, and optional cross-lab enhancements.

The core promise is simple: automate tedious game-development workflows without adding new tedium. Every tool should be understandable at a glance, safe to try, powerful enough to scale through presets and profiles, and integrated into the Unity Editor surface where the user naturally needs it.

A tool that technically exists but is hidden, inaccessible, cramped, duplicated, or visually degraded has not met the product promise.

Release model

- **Core package:** Registry, launcher, lab navigation, theme, preferences, scan/result models, documentation conventions, implementation doctrine, shared widgets, and source/UI verification patterns.
- **Standalone lab packages:** Independently useful products such as Colour Lab, Audio Lab, Asset Placement Lab, Texture Lab, Scene Workflow Lab, Asset Preview and Export Lab, Project Audit and Authoring Lab, Content Generation Lab, UI and Feedback Lab, and future Environment, Settings, Action, Agent, Appearance, and Visualization labs.
- **Bundle package:** Installs all labs and unlocks richer cross-lab linking, dashboards, context menus, graph/canvas handoffs, coordinated workflows, and optional visual dashboards.
- **Optional bridge packages:** Connect two labs or third-party packages. Bridges must remain removable and gracefully degraded when absent.
- **Project adapters:** Thin game-specific wrappers outside the generic package. They may reference game classes; generic packages must not.

Definition of a PungentFunk utility

A utility qualifies for the generic package only if it:

- solves a broadly reusable Unity workflow problem;
- uses generic names, generic data models, stable IDs, and configurable presets;
- works in a clean project or clearly declares optional dependencies;
- provides preview or dry-run before destructive changes;
- records Undo where scene or asset changes are made;
- uses shared visual language and persistence helpers unless deliberately exempt;
- chooses an appropriate Unity Editor integration point rather than forcing every workflow into one window;
- remains visible, reachable, and usable in the intended UI after refactors and cleanup passes.

2. Product Architecture

Package layers

Layer	Purpose	Rules
PungentFunk Utilities Core	Shared shell, registry, control panel, lab menus, theme, preferences, package status metadata, shared scan/result models, documentation patterns, design-system tokens.	Keep Core small, stable, conservative, and boring. No Core-to-lab dependencies.
Runtime model assemblies	ScriptableObjects, MonoBehaviours, data	No UnityEditor references. No editor-only menu logic.

Layer	Purpose	Rules
	models, adapters, and runtime services used outside the Unity Editor.	Use stable IDs, version fields, migration-safe data.
Editor lab assemblies	Editor windows, inspectors, scanners, generators, validators, TreeViews, Scene GUI helpers, overlays, graph tools, authoring panels.	May depend on UnityEditor and Core. Scan/apply operations must be explicit, cancelable where expensive, and cached. Drawing code should not own mutation logic.
Optional bridges	Code connecting labs or third-party packages.	Use version defines, reflection, asmdef constraints, isolated bridge packages, package metadata, or registry/service discovery.
Project adapters	Game-specific wrappers and presets outside generic packages.	Thin only. Generic packages must never compile against them.

Dependency direction

Allowed:

- Lab -> Core
- Editor -> Runtime
- Project Adapter -> Generic Lab
- Bridge -> Lab A + Lab B

Discouraged:

- Lab A directly compiling against Lab B unless Lab B is a declared required dependency.

Forbidden:

- Core -> Lab
- Runtime -> Editor
- Generic Lab -> game-specific project class
- Cyclic lab dependencies
- Accidental compile-time dependencies on optional labs

Shared abstraction rule

A helper belongs in Core only when at least two labs need it and its API is stable enough to support publicly. Immature helpers should stay local until repeated need proves they are truly shared.

3. Source-Truth and Verifiability Doctrine

This section is the corrective layer for repeated cases where a cleanup summary described intent while the actual UI remained worse, hidden, duplicated, cramped, or inconsistent.

Current files outrank all debriefs

A source archive or current repository checkout is the source of truth for implementation state. Debriefs, historical briefs, and conversation summaries describe intent, plan, or claimed outcome. They do not prove active implementation.

Agents and reviewers must distinguish:

- **Stated intent:** What the brief asked for.
- **Claimed outcome:** What the debrief says was done.
- **Source reality:** What current files actually contain.
- **Active UI reality:** What the user can actually see, reach, and use in Unity.

If these disagree, source reality and active UI reality win.

Evidence levels

Evidence level	Acceptable for	Not sufficient for
User current instruction	Intent, priority, override	Claiming implementation completed
Current source file	Source implementation	UI visibility/usability
Active UI screenshot or manual Unity test	UI reachability and layout	Code architecture correctness
Snapshot audit	Package state at audit time	Current state after a newer archive
Debrief or summary	Rationale, claimed goals	Implementation verification
Historical chat	Context and motivation	Current status

No invisible implementation rule

A feature is not functionally implemented if it only exists in code. For UI-facing utilities, implementation requires:

- a current source path;
- an active call path;
- a registered access surface if relevant;
- a visible and reachable UI location;
- usable controls at representative window sizes;
- validation of the intended interaction;
- no contradictory duplicate stale version elsewhere.

Claim evidence format

When an agent claims that a UI section was moved, restored, consolidated, or fixed, it should provide evidence in this form:

- **Feature/control:** The exact feature or section.
- **Previous location:** Where it was before, if known.
- **Current source owner:** File, class, method, and field/state owner.
- **Current active UI path:** Menu/window/tab/panel/foldout/visibility condition.
- **Reachability condition:** What must be selected, toggled, or installed.
- **Validation performed:** Opened window, resized, toggled filters, clicked action, verified output.
- **Known caveat:** Any condition under which the claim does not hold.

A summary without this level of evidence should be treated as unverified.

Active call path requirement

For UI work, reviewers should verify not only that a method exists, but that it is called by the active rendering path. Dead helpers, old panels, hidden foldouts, obsolete tabs, and duplicate static draw methods do not satisfy a UI claim.

Use this review chain:

1. Menu or launcher entry opens the expected window.
2. Window state selects the expected tab/panel/module.
3. The active draw/create method includes the expected section.
4. The section uses current data/state, not stale duplicates.
5. The control is visible and interactable under normal conditions.
6. The action produces or changes the expected state.

4. UI Preservation and No-Hiding Doctrine

UI refactors must improve organization without removing the user's ability to perform the workflow.

The no-hiding rule

Do not solve a UI issue by hiding functionality users still need. Hiding is not the same as simplifying. A feature may be collapsed, moved, filtered, or placed behind Developer Mode only when the new access path is intentional, documented, discoverable, and appropriate to its audience.

A hidden feature is a regression if:

- the user previously relied on it;
- it remains part of the intended product scope;

- no replacement path exists;
- it moved to a less discoverable location without a redirect;
- it is only reachable through an obscure toggle;
- it is technically present but visually clipped, cramped, or below unusable scroll depth;
- it appears in multiple places and the active one is unclear.

UI relocation contract

When moving or consolidating UI controls, the pass must preserve a clear contract:

- **Before map:** List each existing visible control/section and where it lives.
- **After map:** List its new location and access path.
- **Rationale:** Explain why the new location better matches user workflow.
- **Redirect:** Provide a related-tool link, inline handoff, or clear label if the old place no longer hosts it.
- **Visibility:** Ensure the new location is visible at practical window sizes.
- **Persistence:** Preserve relevant user preferences, splitter widths, selected tab, search text, filters, and foldout state.
- **Validation:** Test the moved control in the active window after compile/domain reload.

UI control inventory gate

Before any cleanup pass on a window, create a control inventory:

- window title and primary entry point;
- tabs/modules/foldouts;
- visible sections;
- actionable controls;
- filters/search controls;
- dangerous actions;
- diagnostic/status panels;
- developer-only controls;
- documentation/help links;
- optional integration states.

After the cleanup pass, compare the inventory. Every control should be one of:

- preserved in place;
- moved with a documented new path;
- replaced by a better equivalent;
- intentionally removed with user-approved rationale;
- gated behind a documented mode such as Developer Mode.

Anything else is a regression.

Duplicate and stale UI rule

Duplicated UI can be worse than missing UI. If two panels appear to expose the same feature but only one is active or current, users and agents will misread the system. When consolidating:

- remove or clearly mark stale panels;
- keep compatibility aliases only at access-surface level, not as competing full implementations;
- route old entries to the new active implementation;
- avoid identical labels for different behavior;
- include a deprecation warning only when the old path still exists.

Cramped UI is not completed UI

A feature does not count as restored if the controls exist but are squeezed into unreadable columns, clipped labels, tiny hit targets, or scroll traps. A UI pass must validate:

- narrow width;
- comfortable width;
- tall and short window heights;
- docked and floating mode;
- relevant foldout/tab states;
- empty, partial, and populated data states.

5. Editor UI and UX Principles

The suite should have a cohesive visual identity, but not a rigid one. Consistency should come from shared theme colours, typography rules, spacing scale, button/status/box styles, icons, interaction conventions, safety patterns, registry metadata, lab navigation, and reusable widgets. Consistency should not mean every workflow is forced into the same vertical inspector layout.

Unity-native alignment

PungentFunk tools should feel native to the Unity Editor while retaining a recognisable PungentFunk identity. Unity-native design alignment should influence component semantics, interaction states, authoring flows, windowing, overlays, messaging, iconography, typography, content organisation, accessibility, and keyboard/focus behaviour.

Use Unity-standard semantics first:

- buttons trigger actions;
- toggles enable states;
- dropdowns choose one option;
- radio groups choose mutually exclusive modes;
- sliders adjust continuous ranges;

- numeric fields set exact values;
- object fields reference assets or objects;
- search fields filter;
- tabs separate workflows;
- toolbars collect compact actions;
- progress bars report long work;
- help boxes explain state.

Responsive layout rules

- The window must remain usable at small sizes.
- Prefer vertical scrolling over horizontal scrolling.
- Use wrapping rows, compact cards, collapsible sections, tabs, paging, and column stacking before horizontal scrolling.
- Use draggable splitters where panel proportions matter, and persist sizes.
- Avoid dead space, jumpy controls, and important actions buried below large scroll regions.
- Avoid pretending a UI is responsive because it technically draws; responsive means key controls remain discoverable and practical.

Visual information design

- Use UtilityWindowTheme for shared IMGUI chrome.
- Use the same design tokens for UI Toolkit tools.
- Use colour intentionally for status, category, severity, selection, lab identity, or risk.
- Do not use colour as the only differentiator.
- Use icons, tags, badges, count chips, legends, and labels for repeated states.
- Use diagrams, mini graphs, swatch grids, matrices, timelines, heatmaps, and graph canvases only where they clarify faster than text.

6. Window and Panel Layout Doctrine

Each window and panel should be designed deliberately. A utility layout is part of the product design, not decoration.

Layout templates

Template	Best for	Doctrine
Simple vertical stack	Small settings and single-purpose helpers.	Header -> configuration -> action -> status/help. Avoid for dense or comparison-heavy workflows.
Two-column layout	Source/result, configuration/preview, selection/detail.	Use persistent draggable splitters where both sides compete for space.

Template	Best for	Doctrine
Three-panel layout	Advanced authoring, asset browsers, debug tools, lab hubs.	Left navigation/source, centre workspace/results, right details/actions. Collapse or stack at narrow sizes.
Grid/card layout	Asset sets, palettes, icons, prefab previews, generated names, tool launchers.	Cards should wrap and use compact metadata.
Table/matrix layout	Coverage tools, validation, dependency maps, compatibility checks.	Use sorting, filtering, grouping, and detail panes. Horizontal scroll is acceptable when preserving column relationships matters.
Canvas/lab workspace	Generation, placement, spatial workflows, graph-like relationships.	Central workspace, side configuration, contextual toolbar, mode-specific controls.
Wizard/step flow	Setup, migration, export, risky multi-stage operations.	Show step, inputs, validation, dry-run results, and final apply/export.
Dashboard/overview	Control panel, lab home, health summaries.	Surface summaries, warnings, search, recent tools, quick actions. Do not turn dashboards into dense control panels.
Contextual mini panel	Component, scene selection, terrain, or asset quick actions.	Keep compact and include an Open Full Tool handoff for complex work.

Layout anti-patterns

Avoid:

- long undifferentiated walls of fields;
- excessive nested foldouts;
- large empty gaps from rigid sizing;
- avoidable horizontal scrollbars;
- controls that jump during refresh;
- important actions hidden below large scroll regions;
- duplicate buttons doing the same thing;
- making every utility look identical at the cost of usability;
- using graph/canvas/wizard UI only because it looks advanced;

- claiming a section was restored when it is only present in code but not reachable in the active UI.

Shared resizable panel contract

Every utility window that uses split panels, draggable borders, or multiple stacked sections should use a shared layout contract rather than inventing its own resize math. A custom local splitter is acceptable only when the tool has a genuinely special canvas, graph, matrix, timeline, or preview requirement and documents why the shared contract does not apply.

A resizable panel is valid only when all of the following are true:

- **Handle orientation matches motion.** A vertical divider between left/right panels changes width with horizontal mouse movement. A horizontal divider between top/bottom sections changes height with vertical mouse movement.
- **Drag direction is intuitive.** Dragging a left/right divider to the right expands the panel on the left and reduces the panel on the right, unless the handle explicitly belongs to a right-anchored rail and the invertDelta behavior is named and documented. Dragging a top/bottom divider downward expands the panel above and reduces the panel below.
- **Handle placement matches ownership.** The resize handle is drawn exactly between the two regions it resizes. It should not visually belong to a section it does not resize, and it should not sit detached from both panels.
- **Panel edges stick to handles.** A panel whose size is controlled by a handle should visually attach to that handle. It should stretch to fill the free space between its outer window edge and the handle, with no unowned gap.
- **The remaining panel absorbs remaining space.** In a split layout, at least one region must be flexible and fill the leftover width/height after fixed panels, handles, padding, and toolbar/header/footer regions are allocated.
- **Bounds are relative, not arbitrary.** Minimum and maximum sizes must be derived from the current available window area and the neighboring panel's minimum size. Hard-coded maxima are acceptable only as preferred sizes, not as the final clamp when the window changes.
- **Sizes are clamped before drawing.** Saved splitter values must be clamped on every layout pass before drawing panels. Domain reloads, monitor changes, docking, and window resizing must not restore impossible sizes.
- **Handles remain discoverable.** Drag targets should be large enough to hit comfortably, use a consistent tint/cursor affordance, and have a tooltip when appropriate.
- **Handles do not steal layout space silently.** The handle thickness and spacing should be included in the layout calculation. Never assume a handle has zero width/height.
- **No inverted border regression.** Any new or modified splitter must be validated by dragging in both directions and confirming the expected panel expands/collapses.

Canonical split calculation rule

For a horizontal split of left/right panels, calculate from the current available content width:

1. Reserve fixed outer padding, inner spacing, and the handle width.
2. Compute minLeft, minRight, maxLeft = available - minRight - handle - spacing.
3. Clamp the saved left width between minLeft and maxLeft.
4. Draw left panel at the clamped width.
5. Draw the handle immediately after the left panel.
6. Draw the right panel with ExpandWidth or equivalent so it consumes all remaining space.

For a right-anchored rail, either store rightWidth and draw the flexible content first, or use an explicitly named inverted handle. Do not mix left-owned state with right-owned visual behavior.

For a vertical split of top/bottom sections, use the same rule with height: reserve fixed header/footer/toolbar regions, clamp the top height against the bottom section's minimum height, draw the top section, then the handle, then a bottom section that expands vertically.

Panel stretching and anchoring rules

Panels should stretch according to their job:

- **Navigation/source rails** usually have a clamped width and full available height.
- **Main workspaces/results panels** should expand in both directions and should be the primary absorber of remaining space.
- **Inspector/detail rails** may have a clamped width but should stretch vertically.
- **Configuration sections** may have preferred heights, but results, previews, logs, and dense lists should receive the leftover height.
- **Footer/action bars** should remain visible and should not be pushed below an unrelated scroll region unless the whole workflow is explicitly scroll-first.
- **Scroll regions** should have intentional ownership. Avoid nesting scroll views unless the inner scroll is a matrix, list, table, preview strip, or log with independent navigation needs.

A panel is incomplete if it only draws at its preferred size and leaves unused space below it while sibling panels need room. Use vertical expansion for panels that contain results, lists, previews, diagnostics, documentation, or long-form content.

Adaptive row wrapping contract

Toolbars, filters, chip rows, and compact control rows should adapt to available width before resorting to horizontal scrolling.

Use these rules:

- Controls that form one conceptual row should remain in the same row when there is room.
- Separate semantic rows should stay separate. Do not merge unrelated filter controls, action buttons, and status chips into one crowded row merely because wrapping code exists.

- Rows should wrap by whole control groups, not by splitting labels away from fields or icons away from labels.
- Each control group should have a minimum, preferred, and maximum width. Flexible text/search fields may expand; fixed action buttons should not grow excessively.
- When a row wraps, preserve left-to-right reading order and keep primary actions discoverable.
- Horizontal scrollbars are a last resort for normal controls. They are acceptable for inherently horizontal artifacts such as palette preset strips, timelines, matrices, comparison tables, node canvases, graph views, and texture/preview filmstrips.
- Wrapping should not change the meaning of controls. A row that becomes two rows at narrow width should still preserve grouping, labels, tooltip behavior, and tab/focus order.

Resize and layout validation grid

Every new or refactored utility window should be visually checked in these states:

State	Required result
Narrow docked window	Core workflow remains discoverable; rows wrap before clipping; no essential action disappears.
Comfortable docked window	Panels use available space; no large dead zones; splitters feel natural.
Wide/floating window	Main workspace expands; side rails do not balloon beyond useful maximums unless designed to.
Short window height	Primary actions and current state remain reachable; results/previews scroll intentionally.
Tall window height	Results, docs, previews, or workspaces stretch vertically instead of leaving blank space.
Domain reload	Splitter values, selected tab/module, search/filter text, developer filters, help topic, and foldout state restore and clamp safely.
Empty state	Empty panels show an action-oriented message rather than collapsing to unusable height.
Populated state	Dense content remains scrollable, grouped, and readable.

Common UI implementation failure matrix

These issues are not feature design changes; they are package-wide implementation failures that should be prevented by shared helpers, review gates, and Codex briefs.

Failure	Symptom	Prevention doctrine
Inverted resize handle	Dragging right shrinks the visually left-owned panel, or dragging down shrinks the visually top-owned panel.	Require handle ownership naming and drag-direction validation. Use explicit invertDelta only for right/bottom anchored rails.
Missing resize handle	Two panels compete for space but cannot be adjusted.	Any meaningful split layout needs a visible, persistent, discoverable handle unless the layout is intentionally fixed.
Unconstrained panel	A saved width/height grows beyond the current window and crushes sibling panels.	Clamp against current available size and sibling minimums before drawing every frame.
Detached panel edge	A panel does not stick to the handle or window edge, creating ambiguous free space.	Draw panels and handles in strict visual order; let the flexible panel absorb remainder.
Non-stretching results/details	Results, docs, or preview panels occupy only preferred height while the window has unused vertical space.	Assign expansion to the main workspace/result/detail region; reserve fixed heights only for headers/toolbars.
Over-crowded toolbar	Buttons, chips, search fields, and help icons fight for one line and clip labels.	Use adaptive row groups with min/preferred/max widths and semantic wrapping.
Horizontal scrollbar abuse	Normal filter/action controls require horizontal scrolling.	Wrap or stack controls; reserve horizontal scroll for inherently horizontal content.
Nested scroll trap	User scrolls the wrong region or cannot reach actions reliably.	Use one primary scroll region per panel unless the inner scroll has a distinct data-navigation purpose.
Help affordance crowding	Repeated [?] buttons disrupt headers, rows, or visual rhythm.	Use a consistent compact help affordance in the section header or a contextual help rail; do not insert noisy icons

Failure	Symptom	Prevention doctrine
		beside every label.
Context-losing help	Help opens a generic browser without selecting the relevant utility, section, or topic.	Carry utility ID, section ID, topic ID, and selected context into the help/documentation surface.
Broken persistence	Selected help topic, search text, developer filters, foldouts, splitter widths, and selected tabs reset on reload.	Persist view state with stable per-window/per-section keys and clamp values after reload.
Hidden developer tools	Authoring controls exist but require obscure conditions or are absent from Developer Mode filters.	Register visibility model and show developer/internal/archived utilities through explicit Developer Mode filters.
Dead helper syndrome	A correct-looking draw/helper method exists but is not called by the active UI path.	Require active call path proof for all UI claims; remove stale helpers or mark them clearly.
Debrief/source mismatch	Summary says moved/restored, but code or active UI contradicts it.	Use claim evidence format and never mark UI work complete without source owner + active path + validation.
Core-to-lab dependency leak	Utility Core references Notes, Project Audit, Debug, Palette Designer, or other labs directly.	Core may hold registry/service abstractions only; lab-specific code belongs in lab or bridge assemblies.
Repaint scanning	Script indexing, reflection, or documentation generation runs from OnGUI/draw paths.	Draw current state only; run scans/indexing/doc generation through explicit, cached, staged services.
Generated docs overwrite curated docs	Automated index replaces developer-authored descriptions or notes.	Generated docs may fill missing drafts or separate generated sections; curated/manual content wins unless the user explicitly overwrites it.
Inherited-method noise	API index lists EditorWindow, MonoBehaviour, object, or Unity lifecycle methods as	Index declared public package APIs by default; inherited/lifecycle methods are hidden or grouped separately unless explicitly

Failure	Symptom	Prevention doctrine
	primary APIs.	relevant.
Duplicate documentation sources	Notes, docs links, generated indexes, and built-in topics disagree.	Establish precedence: curated lab docs > curated utility notes > generated supplement > stale snapshot/debrief. Surface source labels.

Help, documentation, and generated-content layout rules

Documentation and help tools should follow the same UI layout standards as every other panel, with additional context-preservation rules:

- Help buttons should be visually calm. Prefer one help affordance per section or header cluster, not a noisy button after every field.
- Help affordances must not force headers, filters, or action rows to wrap prematurely. If a header is crowded, move help into a compact right-aligned icon slot, contextual side rail, or overflow menu.
- Opening help from a utility must preserve context: utility ID, tab/module, section, selected item if relevant, and topic anchor.
- Generated scripting references should list useful package-authored APIs first. Inherited Unity/Object methods and lifecycle noise should be collapsed, filtered, or placed in a secondary “inherited/lifecycle” group.
- Generated documentation must not overwrite curated documentation by default. It may create missing draft descriptions, suggest updates, or write to clearly generated sections/files.
- Documentation surfaces should label source type: curated, generated, imported, stale, missing, or draft.
- If multiple documentation sources disagree, the UI should surface precedence rather than silently merging contradictory content.

Recommended shared helper scope

The recurring failures above should be treated as evidence that splitters, wrapping rows, help affordances, and documentation context handoffs belong in shared Core helpers once the APIs are stable.

Recommended shared helpers include:

- ResizableSplitLayout or equivalent: clamps widths/heights using available size, neighboring minimums, handle thickness, and anchor ownership.
- AdaptiveToolbarRow or equivalent: groups controls by semantic row, wraps whole groups, and avoids horizontal scrolling for ordinary controls.
- StretchPanelScope or equivalent: draws a themed panel that can claim leftover vertical/horizontal space intentionally.

- ContextualHelpButton or equivalent: stores utility ID, section ID, and topic ID without crowding row layout.
- DocumentationSourceBadge or equivalent: labels curated/generated/stale/draft documentation sources.
- UiStatePersistenceKeys or equivalent: centralizes stable key naming for splitters, searches, filters, selected tabs, selected help topic, developer mode filters, and foldouts.

Do not turn these helpers into a rigid visual template. They should guarantee correct sizing, persistence, and interaction behavior while still allowing each lab to choose the layout pattern that suits its workflow.

7. Progressive Disclosure, Assistance, Accessibility, and Messaging

Tools should be understandable at a glance and deep enough for advanced users. Progressive disclosure is not a license to hide essential workflow controls.

Progressive disclosure rules

- Show the core workflow first.
- Collapse advanced options by default only if the core workflow remains complete.
- Do not bury essential controls under advanced sections.
- Do not move a required control into Developer Mode unless it is truly developer-only.
- If a feature is filtered out, provide a visible filter state and a way to reveal it.
- If a feature is archived or hidden, provide restore or show-hidden controls where appropriate.

Tooltips and contextual help

- Every meaningful field should have a tooltip or help affordance.
- Tooltips should explain what the field controls, when to change it, how it differs from related options, and whether the value is destructive, experimental, expensive, or preview-only.
- Complex enum choices should have a nearby description.
- Labels should describe the enabled state or action clearly.

Accessibility expectations

- Do not rely on colour alone.
- Pair colour with icons, labels, shapes, patterns, or tooltips.
- Maintain readable contrast.
- Provide visible focus states where feasible.
- Preserve logical keyboard navigation where the UI technology supports it.
- Avoid tiny click targets for frequent actions.
- Use plain-language labels for common operations.
- Runtime UI and Feedback Lab components should expose meaningful accessibility hierarchy, labels, roles, values, and state where supported.

Empty states and errors

Empty states should explain what is missing and offer one clear next action. They should not use error styling unless something failed.

Errors should state:

- the problem;
- likely cause;
- next useful step.

Warnings should identify risk before changes are applied. Info messages should confirm state without blocking the workflow.

8. Unity Editor Implementation Doctrine

PungentFunk tools should choose Unity Editor implementation patterns deliberately. This is a selection framework, not a mandate to rewrite stable tools.

IMGUI, UI Toolkit, and migration policy

- IMGUI remains acceptable for existing windows, small utility panels, debug tools, compatibility wrappers, and internal workflows where code-driven rendering is fastest and clearest.
- UI Toolkit should be evaluated first for new flagship lab windows, persistent complex workspaces, product-facing tools, rich styling, component-rich views, retained state, UXML/USS-driven presentation, and tools expected to grow significantly.
- Do not rewrite for fashion. Rewrite when there is a concrete product, maintainability, accessibility, performance, or UX reason.
- Hybrid strategies are acceptable.
- Major tools should document why they use IMGUI, UI Toolkit, CustomEditor, Overlay, TreeView, graph/canvas, or a hybrid pattern.

Implementation role selection

Role	Use when	Doctrine
EditorWindow / Lab Window	Workflow has filters, previews, generated results, scan/apply stages, dashboards, tabs, or persistent workspace.	Keep scan/apply logic outside repaint. Cache expensive state.
CustomEditor	Tool belongs directly to a selected component or asset.	Keep compact. Provide validation, setup helpers, and Open in Lab handoffs.
PropertyDrawer	A serializable field type needs reusable presentation.	Use for field-level UX, not whole workflows.
Scene GUI / Handles	User edits spatial data	Keep lightweight. Respect

Role	Use when	Doctrine
	directly in the Scene View.	Undo, snapping, visibility, and Prefab Mode.
Scene View Overlay / EditorTool	Persistent compact controls are needed while working in Scene View.	Use for placement, navigation, debug toggles, paths, terrain/surface, and environment previews.
TreeView / MultiColumnView	Data is hierarchical, dense, expandable, or audit-like.	Preserve row identity, selection, expansion, sorting, filtering, and scroll position.
Search Provider / Query Tool	Dataset is large and users think in terms of discovery/filtering.	Avoid inventing custom query syntax unless the lab needs it.
Shortcut / Command	Action is frequent, safe, contextual, and easy to describe.	Avoid rare or destructive shortcuts.
Context Menu	Short action is most useful from selection.	Keep concise and route full workflows to labs.
Graph / Canvas Tool	Relationships, branching, dependencies, or pipelines are the primary authoring problem.	Use canvas only where it clarifies more than forms, lists, or tables.
Optional Bridge	Feature links labs or optional packages.	Keep removable and gracefully degraded.

9. Safety and Performance Doctrine

Drawing must not own mutation

Editor drawing methods should render current state. They should not own expensive scans, asset writes, scene mutation, graph mutation, broad reflection work, or project-wide operations.

Use this flow for broad, destructive, or expensive operations:

1. Configure.
2. Scan or validate.
3. Preview.
4. Apply selected or apply all.
5. Summarize changed, skipped, and failed items.

Performance standards

- Do not scan all scenes or assets on every visual refresh.

- Cache scan results and rebuild only when settings change or the user refreshes.
- Use staged editor work for long preview, export, bake, icon, or scan operations.
- Use cancelable progress for expensive work where practical.
- Avoid Thread.Sleep in editor tools.
- Avoid broad SceneView repaint unless required.
- Preserve row/node identity in large lists, trees, tables, and graphs.
- For UI Toolkit, use virtualization/pooling where large repeated lists are involved.
- Static state is a cache, not source-of-truth user data.

Safety standards

- Preview or dry-run before destructive and batch operations.
- Default dangerous toggles to safe behavior.
- Do not overwrite user data without explicit consent.
- Do not modify shared materials, prefab assets, imported assets, or scenes by default without warning.
- Show a summary of changed, ignored, and failed items.
- Be explicit about scope: selection, active scene, open scenes, prefab assets, project assets, generated groups, or all matching files.

10. Editor Access and Integration Principles

Menu items are the baseline access method, not the only access method. Tools should appear where the user naturally needs them.

Access method	Principle
Organised menus	Use clean, curated menu roots. Avoid duplicate clutter and old project-specific names except as compatibility aliases.
Utility Control Panel / Lab Hub	Every significant utility should register with the central registry and be searchable/filterable.
Toolbar / command access	Use sparingly for frequent global or status-like actions. Do not mirror the full menu tree.
Scene View overlays	Use for placement, surface alignment, navigation, gizmo browsing, debug visual toggles, path tools, terrain/surface helpers, and environment previews.
Context menus	Use for concise selected-object, asset, folder, material, prefab, texture, terrain, palette, or ScriptableObject actions.
Inspector integration	Use compact toolbars, validation panels, quick actions, dependency messages, and

Access method	Principle
	Open in Lab buttons.
Cross-utility embedding	Use optional services, registry lookup, package detection, reflection-safe adapters, or bridge packages.
Shortcuts	Use for frequent, safe, easy-to-describe commands.
Context-aware launch	Preload relevant selection context and fall back gracefully.
Recent/pinned/resume	Preserve recent tools, favourite labs, last active module, and related-tool handoffs through shared preferences.

Access-surface preservation

Every utility should have a documented primary access path. If an access path changes, preserve a compatibility route or visible handoff until users can find the new location. Do not leave orphan menu items opening stale windows, and do not remove an access path during cleanup unless the new path is obvious or explicitly approved.

11. Registry, Navigation, and Lab Hubs

The registry is the package-level discovery contract. Every significant utility should have complete metadata.

Field	Purpose
Id	Stable machine-readable identifier. Use kebab-case. Never reuse for a different tool.
DisplayName	User-facing, generic, clear, asset-store friendly name.
Lab / Module / Category	Lab is product area, module is subdomain, category supports menu/search taxonomy.
Description	One-sentence promise: what the tool does and when to use it.
PackageStatus	Stable, Experimental, In Progress, Deprecated, or Project Adapter.
ImplementationRole	EditorWindow, CustomEditor, PropertyDrawer, TreeView, Scene GUI, Overlay, Runtime Service, Bridge, Graph Tool, or hybrid.
RelatedUtilityIds	Complementary tool links. Must not imply hard dependency unless declared.
CapabilityFlags	Selection, context menu, scene overlay, batch

Field	Purpose
	action, lab hub, tree data, scene handles, inspector embedding, graph workspace, search, shortcut, accessibility metadata, package bridge.
VisibilityModel	Normal, Developer Mode, Hidden/Internal, Deprecated, Missing Dependency, or Project Adapter.
ActiveUiPath	The active menu/window/tab/panel/foldout where the user reaches the feature.

Menu policy

- Primary editor menus should use one chosen root consistently for the release line.
- Asset context menus should use Assets/PungentFunk Utilities/....
- GameObject context menus should use GameObject/PungentFunk Utilities/....
- CreateAssetMenu paths should use PungentFunk Utilities/[Lab]/[Asset Type].
- Legacy aliases are allowed only as compatibility routes and should be marked internally.

Visibility model rules

If a utility is hidden, archived, developer-only, internal, or dependency-gated, the registry should say so. The Control Panel should allow appropriate filtering for Developer Mode and archived/internal utilities. Hidden status must not be used to paper over unfinished, broken, or hard-to-layout UI.

12. Optional Dependency and Cross-Utility Integration Principles

PungentFunk Utilities should become more powerful when multiple labs are installed together without making standalone labs fragile.

Dependency levels

Level	Meaning
Required dependency	Must exist for the package to compile and function. Keep minimal and documented.
Optional integration	Activates when another lab or package is installed. Must not compile-break when absent.
Suggested companion	Improves workflow but is not required. Missing companion messaging should be calm and non-blocking.

Optional integration techniques

Use:

- assembly definition optional references;
- version defines;
- define constraints;
- reflection-safe discovery;
- interface packages;
- adapter classes in separate integration folders;
- registry-based service lookup;
- loose ScriptableObject references;
- conditional compilation;
- soft dependency metadata in descriptors.

Cross-lab dependencies are product decisions, not accidental using directives.

13. Runtime, Editor, Data, and Serialization Rules

Runtime package rules

- Runtime scripts must compile in player builds without UnityEditor.
- Runtime ScriptableObjects should contain data and pure runtime logic only.
- Do not store editor-window state in runtime assets unless it has runtime meaning.
- Serialized fields should have tooltips and sane defaults.
- Use stable IDs or GUID-like keys for references across assets, adapters, graphs, or packages.

Editor package rules

- Editor windows should use SerializedObject and SerializedProperty when editing serialized assets/components where Undo, prefab overrides, or multi-object editing matters.
- Use Undo for scene/object changes.
- Use AssetDatabase and EditorUtility.SetDirty safely.
- Prompt before opening/changing scenes, editing prefab assets, or applying destructive project-wide changes.
- Separate rendering from scanning, asset mutation, scene mutation, graph mutation, and service logic.

Serialization and prefab doctrine

Prefab-facing tools must distinguish prefab assets, prefab instances, variants, nested prefabs, Prefab Mode isolation, and in-context prefab editing. Do not silently apply scene-instance decisions to prefab assets. Generated assets should preserve GUID/meta stability where possible and avoid delete/recreate churn when update-in-place is safe.

14. Asset, Package, and Import Pipeline Doctrine

Editor utilities are part of Unity's asset, package, import, and compilation workflows. They must respect those systems.

Package and distribution doctrine

- Design labs so they can ship as standalone products and as a bundle.
- Use package boundaries, asmdefs, samples, documentation, icons, presets, and optional bridges deliberately.
- Keep content organised: runtime code, editor code, samples, documentation, icons, presets, migration helpers.
- Do not rely on project-specific folder paths.
- Default output folders should be configurable, safe, and visible.
- Use special Unity folders deliberately and sparingly.

AssetDatabase and import workflow doctrine

- AssetDatabase is editor-only.
- Asset operations should be explicit and previewed when they affect many assets.
- Batch operations should avoid unnecessary refresh/import churn.
- Tools should distinguish source assets, generated assets, temporary previews, and user-authored assets.
- Generated asset names should be deterministic where useful, collision-safe, and reversible.

15. Lab Taxonomy

Lab	Product role	Primary ownership
Utility Core	Shared shell and UX infrastructure.	Registry, control panel, lab menus, tray/minimizer, theme, prefs, performance helpers, scan models, status vocabulary, documentation links, design-system tokens, source/UI verification helpers.
Debug Lab	Runtime/editor debug control and signal observation.	Debug router, channels, relay, reflected fields/actions, scheduled calls, router panels, debug integration points.

Lab	Product role	Primary ownership
Asset Placement Lab	Reusable scene asset placement and procedural dressing.	Asset sets, rules, scatter/grid/socket/group workflows, placement results, scene application, handles, overlays, spatial previews.
Scene Workflow Lab	Scene authoring speed tools.	Scene navigation, gizmo browsing/sources, surface alignment, path authoring, Scene View assistance.
Colour Lab	Palette creation, analysis, accessibility, and application.	Palette assets, swatches, harmony, contrast, deficiency preview, generation, material/UI application.
Texture Lab	Procedural texture/mask generation and array baking.	Texture settings, generators, texture arrays, bake validation, preview surfaces.
Audio Lab	Surface/contact audio data and coverage validation.	Audio materials, clip sets, profiles/matrices, terrain mappings, contact router/classifier, coverage scanners.
Asset Preview and Export Lab	Preview/render/export support.	Prefab icons, prefab extraction, font previews, output packaging, preview queues.
Project Audit and Authoring Lab	Generic diagnostics and batch tools.	Reference assignment, terrain usage, tuning copy, rename, coverage matrix, token validation, notes/roadmap, inventories.
Content Generation Lab	Reusable text/name generation.	Name lists, MadLib lists, generator window, token/content preview/export flows.
UI and Feedback Lab	Reusable prompt and feedback runtime/editor tools.	Input prompt tooling first; future presenters remain layout-agnostic and accessibility-aware.
Environment Simulation Lab	Reusable environment simulation services.	Wind, weather, time/calendar, surface conditions, local volumes,

Lab	Product role	Primary ownership
		debug previews, optional render-pipeline adapters.
Save and Settings Lab	Reusable settings and persistence infrastructure.	Settings profiles, JSON storage, save slots, input binding overrides, reset/default/versioning.
Action / Ability Authoring Lab	Generic action authoring.	Action sequences, conditions, targeting, effects, optional projectile/status/hazard modules only if generic.
Agent Simulation Lab	Generic AI/simulation authoring.	Utility evaluators, state assets, needs/traits/emotions, interaction scoring, social groups.
Appearance Lab	Generic appearance authoring.	Appearance option assets, catalogues, wearable attachments, colour/material options, validators.
Visualization / Graph Lab	Shared visualization widgets and graph/canvas infrastructure.	Matrices, heatmaps, graphs, canvas workspaces, minimaps, blackboards, selected-element inspectors, data-view widgets.

16. Documentation Architecture

The design bible should not carry every task. Documentation should be layered so humans and AI agents can find the correct source quickly.

Required layers

Layer	Artifact	Purpose	Volatility
Vision	docs/design-bible/index.md or vision.md	Short alignment, product promise, labs, reading paths.	Low
Doctrine	docs/design-bible/doctrine.md	Stable architecture, UX, safety, package doctrine.	Low
Patterns	docs/design-bible/patterns/*.md	Concrete reusable implementation patterns.	Medium

Layer	Artifact	Purpose	Volatility
Labs	docs/design-bible/labs/*.md	Per-lab scope, dependencies, workflows, definition of done.	Medium
Agent instructions	.github/copilot-instructions.md, .github/instructions/*.instructions.md, .github/skills/.../SKILL.md	AI behavior, repo rules, path-specific rules, task procedures.	Medium
Snapshots	Documentation/Snapshots/*	Script counts, feature inventory, audit reports.	High
Execution	Issues, PRs, Codex briefs, update-pass notes.	Work planning and implementation deltas.	High
Release	CHANGELOG.md, release notes, tagged PDFs.	Human-readable change history.	Medium

PDF/source rule

Markdown should be the source of truth for doctrine, lab specs, pattern pages, and agent instructions. PDF should be generated for distribution, archival snapshots, or handoff. Do not treat a PDF as the only editable source.

17. AI Task Routing and Failure Prevention Matrix

AI agents should use this matrix before proposing implementation changes.

User request	Likely artifact	First check	Common failure to avoid
Audit current package	Snapshot audit + feature inventory	Current archive/source tree	Using old script counts as current facts.
Improve doctrine	Design bible	Stable principles and companion docs	Adding volatile implementation status to doctrine.
Create Codex brief	Implementation pass brief	Affected files, doctrine, validation	Vague objectives without file targets.
Fix compile error	Targeted code patch	Runtime/editor boundary and	Moving code to wrong assembly to

User request	Likely artifact	First check	Common failure to avoid
		asmdefs	silence error.
Refactor large window	Panel/service/state split	Preserve user workflow and serialized state	Rewriting UI and losing functionality.
Move UI section	UI relocation contract	Before/after control inventory	Claiming it moved without verifying active UI path.
Clean up cramped UI	Layout pass with visual QA	Control inventory + narrow/wide screenshots	Hiding controls to make layout look cleaner.
Fix splitters/resizable panels	Shared resizable panel contract	Handle ownership, current available size, sibling minimums, and drag-direction validation	Inverted handles, unconstrained panels, detached panels, or hard-coded max sizes.
Add adaptive toolbar/filter row	Adaptive row contract	Semantic row grouping, min/preferred/max widths, and narrow-width validation	Clipped rows, accidental semantic merging, or horizontal scrolling for ordinary controls.
Add help/documentation affordances	Contextual help contract	Utility ID, section ID, topic ID, and source precedence	Crowded [?] buttons or generic help windows that lose context.
Generate docs or scripting reference	Documentation generation contract	Curated/manual docs and declared package APIs	Overwriting curated descriptions or indexing inherited Unity/Object noise as primary API.
Add lab feature	Lab spec + implementation pass	Core/lab/bridge boundary	Putting lab-specific logic in Core.
Add cross-lab feature	Optional bridge or soft integration	Dependency direction	Hard compile dependency on optional lab.
Add scanner	Shared scan model/pattern	Explicit scan/cache/apply	Scanning during repaint.
Add scene tool	Scene GUI/Overlay/EditorTool + window handoff	Undo, Prefab Mode, lightweight Scene GUI	Heavy work in OnSceneGUI.

User request	Likely artifact	First check	Common failure to avoid
Add docs	Markdown source + generated PDF if needed	Layer and audience	Treating PDF as only source of truth.

18. AI Output, Briefing, and Debrief Standards

For audit responses

An audit should include:

- current state;
- design bible alignment;
- gaps;
- priority fixes ranked as release blocker, high-value hardening, UX polish, future enhancement;
- recommended implementation pass;
- uncertainty about source freshness;
- source evidence for implementation claims;
- UI evidence requirements for layout claims.

For implementation briefs

A Codex-ready brief should include:

- primary objective;
- relevant doctrine;
- affected files;
- required changes;
- architecture constraints;
- UI/UX constraints;
- safety requirements;
- performance requirements;
- validation steps;
- known limitations;
- output expectations;
- before/after control inventory requirement for UI work;
- active UI path proof requirement for moved/restored sections;
- resizable panel contract requirement for split layouts;
- adaptive row/wrapping requirement for toolbar, filter, and action rows;
- help context preservation requirement for documentation/help affordances;

- curated/generated documentation precedence requirement for documentation automation.

For completed implementation summaries

A completed pass should report:

- summary;
- files changed;
- architecture notes;
- UI/UX notes;
- safety/performance notes;
- validation steps;
- known limitations;
- source evidence for key claims;
- active UI evidence for UI claims;
- explicit mention of anything unverified;
- splitter/resize validation result where panels were touched;
- narrow/comfortable/wide layout validation result where control rows were touched;
- persistence/domain reload validation result where view state was touched.

Debrief truthfulness standard

A debrief must not say “restored”, “moved”, “implemented”, “fixed”, or “completed” unless the claim has been verified against current files and, for UI changes, against the active UI path.

Use safer wording when appropriate:

- “Added source support for…” when UI reachability is unverified.
- “Prepared a new panel method for…” when active draw path is unverified.
- “Moved in code; needs Unity visual QA” when compile/source is done but UI is untested.
- “Intended to replace…” when the old path may still exist.

Canonical examples for agents

Good:

- A scanner caches results, refreshes explicitly, previews changes, applies with Undo, and reports changed/skipped/failed items.
- A cross-lab feature is implemented as an optional bridge or soft registry lookup.
- A large window split preserves current workflow while extracting state, scanning, and rendering into separable units.
- A UI relocation includes before/after control inventory, active call path, visible access path, and narrow/wide validation.
- A split-panel update identifies the handle owner, clamps against sibling minimums, and verifies drag direction.

- A documentation generator writes to generated sections only, preserves curated descriptions, and labels source precedence.

Bad:

- Runtime code references UnityEditor.
- OnGUI triggers a project-wide scan every repaint.
- A quick-fix button silently modifies shared materials or prefab assets.
- A UI cleanup hides useful audit information rather than organizing it.
- A debrief says a panel was restored when the control only exists in an unused method.
- A resize handle is visually between panels but changes the wrong panel or exceeds the available window space.
- A generated scripting index lists inherited Unity/Object methods as the main public API.
- A help button opens a generic browser and loses the utility/section/topic context.
- A generic lab directly imports a project-specific class.

19. Package Hardening Rules

Namespaces

Use package namespaces before release. Recommended roots include:

- PungentFunk.Utilities
- PungentFunk.Utilities.Runtime
- PungentFunk.Utilities.Editor
- PungentFunk.Utilities.Editor.Core
- PungentFunk.Utilities.Editor.Theme
- PungentFunk.Utilities.Placement
- PungentFunk.Utilities.Audio
- PungentFunk.Utilities.Colour
- PungentFunk.Utilities.Generation
- PungentFunk.Utilities.SceneTools
- PungentFunk.Utilities.UI
- PungentFunk.Utilities.Visualization

Assembly definitions

- Create separate runtime and editor asmdefs for each releasable lab or Core + all labs if shipping as a single bundle first.
- Editor asmdefs reference runtime asmdefs.
- Runtime asmdefs never reference editor asmdefs.
- Optional bridges get their own asmdefs and dependencies.

- Third-party dependencies such as TextMeshPro, Input System, Graph Toolkit, or accessibility-facing packages should be declared, optionalized, or isolated.

Documentation standard

Each lab needs:

- overview;
- setup checklist;
- core concepts;
- recipes;
- troubleshooting;
- optional integration notes;
- UI implementation role notes;
- accessibility notes;
- package/dependency notes;
- API/adapters page;
- validation checklist.

Every editor window should have a Help/Notes foldout summarising what the tool changes and what it never changes. Every destructive or batch workflow should document Undo support and limits.

20. Versioning, Review, and Maintenance

Review cadence

Artifact	Review trigger	Suggested owner
Design bible	Doctrine, boundary, UX, package, UI verification, or AI interpretation change.	Project/product owner plus technical reviewer.
Pattern pages	New repeated implementation pattern or major refactor.	Tooling maintainer.
Lab specs	New lab feature, lab split, dependency change, or public workflow change.	Lab owner.
Snapshot audit	Every meaningful script/package update.	Audit tool or implementation owner.
Feature inventory	After feature passes or roadmap changes.	Planning owner.
Agent skill	Agent behavior, output format, task workflow, or verification standard	AI tooling owner.

Artifact	Review trigger	Suggested owner
	changes.	
Changelog	Public behavior, package, or doctrine milestone.	Release owner.

Changelog policy

Use curated prose, not raw commit dumps. Record changes under categories such as Added, Changed, Deprecated, Removed, Fixed, Security, Documentation, and Doctrine. Separate package release changelogs from doctrine changelogs where necessary.

Decision log policy

Use lightweight ADR-style notes for major decisions, such as:

- Core/lab boundary changes;
- IMGUI vs UI Toolkit policy changes;
- optional bridge architecture;
- public menu root changes;
- package splitting strategy;
- AI-agent instruction changes;
- source/UI verification doctrine changes;
- visibility model and Developer Mode gating policy.

21. Utility Definition of Done

A PungentFunk utility update is complete only when the relevant items are satisfied:

- Registered in PungentUtilityRegistry or a consistent per-lab registrar.
- Clean primary menu path and optional context menus.
- Deliberate implementation role documented for major tools.
- UtilityWindowTheme/UtilityWindowPrefs used for IMGUI tools or equivalent design tokens for UI Toolkit.
- Unity-standard component semantics preserved.
- Responsive at small window sizes.
- No avoidable horizontal scrollbar.
- Draggable/persistent splitters where panels compete for space, with verified orientation, drag direction, clamping, and sibling minimums.
- Meaningful tooltips/help for important fields.
- Useful empty states.
- Precise warning/error/info messages.
- Does not rely on colour alone.
- Reuses shared widgets/helpers where patterns repeat.
- Uses dense list/tree/table patterns for dense data.

- Keeps Scene GUI lightweight.
- Uses graph/canvas only when relationships justify it.
- Uses serialized property workflows where needed.
- Runtime code compiles without UnityEditor.
- Editor code lives in Editor folders or editor asmdefs.
- Avoids compile-time dependencies on game-specific classes.
- Optional integrations degrade gracefully.
- AssetDatabase operations are explicit and safe.
- Tolerates domain reload, code reload, window close/reopen, and saved layouts; selected tabs, search text, filters, help topic, developer filters, foldouts, and splitter values restore where relevant.
- Scan-heavy tools cache results and refresh explicitly.
- Batch/destructive operations provide preview/dry-run and Undo where possible.
- Tested in a clean project, with optional labs absent, and with the full bundle installed where relevant.
- UI-facing features have active UI paths, not just source methods.
- UI relocation has a before/after control inventory.
- Moved/restored UI has verified source owner, active draw path, visibility condition, and usable layout.
- Split layouts use the shared resizable panel contract or document a justified exception.
- Toolbar/filter/action rows use adaptive wrapping by semantic groups and avoid horizontal scrolling unless the content is inherently horizontal.
- Help affordances preserve context without crowding headers or control rows.
- Generated documentation preserves curated content, labels generated sections, and filters inherited-method noise by default.
- AI-facing briefs or summaries include architecture notes, UI/UX notes, safety/performance notes, validation steps, known limitations, and explicit unverified items.

22. UI Refactor Definition of Done

For any pass that changes a window, panel, utility browser, debug control center, design validation audit, documentation links window, minimizer, or other visible editor UI, apply this stricter gate.

Required artifacts for UI refactor completion

- Before control inventory.
- After control inventory.
- List of intentionally removed controls.
- List of moved controls with new access paths.
- List of developer-only or hidden controls with rationale.
- Source file/class/method owner for each major section.

- Active draw/create path for each major section.
- Manual Unity validation steps.
- Narrow, comfortable, wide, short-height, and tall-height validation.
- Confirmation that no essential action is only reachable through accidental layout expansion.
- Splitter/handle validation for any touched split layout: owner, orientation, drag direction, min/max clamp, and persistence.
- Adaptive row validation for any touched toolbar/filter/action row: semantic grouping, wrapping order, clipped label check, and horizontal-scroll exception if used.
- Help/context validation for any touched help affordance or documentation link.
- Documentation precedence validation for any generated docs/indexing system.

Refactor acceptance test

A UI refactor is acceptable only when a user can answer these questions without reading source code:

- Where do I start?
- What is selected or filtered?
- What changed after the latest scan/action?
- Where did the controls from the previous version go?
- Which actions are dangerous?
- Which controls are developer-only or hidden?
- How do I restore hidden/archived utilities?
- How do I open the full tool from a compact/contextual surface?

Regression indicators

Treat these as failures:

- The debrief claims a section moved, but the active UI still draws the old placement.
- A needed section exists only in a helper method that is not called.
- A panel is technically visible but usable only at impractically wide sizes.
- Cleanup removed duplication by removing the only discoverable access path.
- A filter hides important tools without a visible filter indicator or restore path.
- A developer-mode section is required for normal user workflows.
- Visual hierarchy improved while task completion got worse.
- A resize handle appears but drags in the wrong direction or allows a panel to crush its sibling.
- A section does not stretch vertically even though it owns results, documentation, previews, diagnostics, or other long-form content.
- A normal toolbar/filter row requires horizontal scrolling instead of semantic wrapping.
- Help buttons crowd polished headers or rows.
- Help/documentation opens without focusing the relevant utility, section, or topic.

- Generated docs overwrite curated descriptions or duplicate another documentation source without precedence labels.

23. Reference Basis

This bible is grounded in:

- the PungentFunk Utilities manual architecture doctrine;
- the AI-agent-tailored design bible and skill framework;
- Unity editor implementation categories including IMGUI, UI Toolkit, custom windows, inspectors, property drawers, TreeView/ListView, Scene View, Gizmos, Handles, overlays, EditorTools, Search, shortcuts, AssetDatabase, Package Manager, assembly definitions, prefabs, domain/code reload, and workspace/windowing;
- Unity Foundations concepts for accessibility, colour, iconography, interactions, typography, UI writing, empty states, errors, messaging, overlays, and windowing;
- project history of repeated utility audits, feature inventories, package-hardening passes, control-panel updates, documentation-link workflows, debug-control-center updates, design-validation-audit passes, and AI-agent update briefs;
- the specific observed failure mode where implementation debriefs overstated active UI reality and cleanup passes hid, cramped, duplicated, or misplaced needed functionality.

24. Closing Doctrine

PungentFunk Utilities should remain modular in code, cohesive in user experience, conservative in dependencies, accessible in presentation, deliberate in Unity Editor implementation choices, and honest about verification. The suite can feel interconnected without becoming tightly coupled. Core provides discovery and shared UX. Labs provide focused capability. Bridges provide optional collaboration. Project adapters preserve game-specific workflows without contaminating generic assets.

When in doubt, choose the smaller generic utility, the thinner adapter, the safer apply flow, the clearer layout, the more explicit dependency boundary, the more accessible communication pattern, the more accurate documentation layer, the more verifiable implementation claim, and the Unity Editor integration point that appears where the user naturally needs the tool.

A tool is not done because a debrief says it is done. A UI section is not restored because a method exists. A cleanup is not successful if it hides what users need. The final standard is current source, active UI, user-visible reachability, and verified workflow integrity.